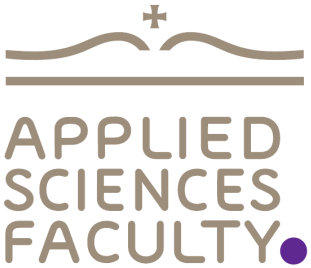
C++ — лекція 25. Уніфікована ініціалізація





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Види ініціалізації в C++98

- C++98 надає цілий зоопарк засобів ініціалізації... Різнорідний.

```
int x = 11;
int y (11);

double dx = 1.23;
int xz = dx;     //! Narrowing. Hope for warning ...

int arr [] = {1,2,3,4,5};
int arr2 []{1,2,3,4,5};

vector<int> v1(arr, arr+sizeof(arr)/sizeof(arr [0]));
                                                //! Pair of iterators
// vector<int> v2 = {1,2,3,4,5};      //! In C++98 --- no way

struct ArrWrapper{
    const int arr [5]; //! No way to init in C++98!
};
const int* darr = new const int[5]; //! No way to init in C++98!
                                     //! Error in GCC, but "OK" in MSVC
const int* darr = new int[5];        //! No way to init in C++98!
int* di = new int(333);
```

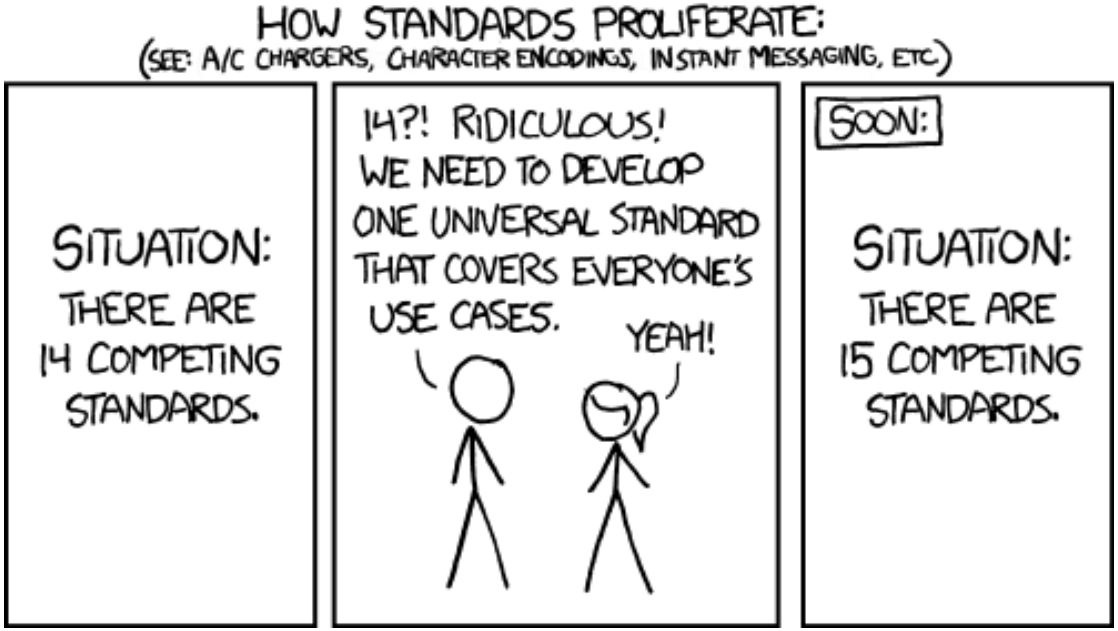
- **class** VictimBase{
  .....
  explicit VictimBase(**int** v=0);
  };

```
// VictimBase vx = 5; //! Not Allowed because of explicit
VictimBase vy(5);

struct Ratio1{
    int num, den;
};
Ratio1 r1a={5,7};
// Ratio1 r1b (5,7); //! Error

struct Ratio2{
    int num, den;
    Ratio2(int c, int z): num(c), den(z) {}
};
// Ratio2 r2a = {3,4}; //! Not allowed in C++98 for classes
                       //  with constructors !
Ratio2 r2b (3,4);
```

- C++11 ввів новий спосіб ініціалізації, придатний для всезагального використання.
- Формально, способів ініціалізації стало більше[1]:



- Однак, за одним винятком, решта не є необхідними, ними можна не користуватися. (Про виняток — трішки пізніше).
- `https://mikelui.io/2019/01/03/seriously-bonkers.html` – "Initialization in C++ is Seriously Bonkers", містить дуже круту гіфку!
- Книга про ініціалізацію в C++: `https://leanpub.com/cppinitbook`, `https://www.cppstories.com/2022/cpp-init-book/`.

[1] https://xkcd.com/927/

- ```
  int x = {11};                    // int x = 11;
  int y {11};                      // int y(11);

  double dx = 1.23;
  // int xz {dx};     //! Error. Narrowing not allowed!

  int arr [] = {1,2,3,4,5};
  int arr2 []{1,2,3,4,5};

  vector<int> v2 = {1,2,3,4,5};
  vector<int> v2a{1,2,3,4,5};

  struct ArrWrapper{
          const int arr [5]{3,4,4,5,6};
          const int arr2 [5]={3,4,4,5,6};
  };
  const int* darr = new const int [5]{1,2,3,2,1};
  const int* darr2 = new int [5]{1,2,3,2,1};
  int* di = new int{333};
  ```

- GCC 4.9 та 5.2 помилково дозволяють скомпілюватися "звужуванню", але дають попередження.
  ```

```
struct Ratio1{
    int num, den;
};
Ratio1 r1a={5,7};    // agregate initialization
Ratio1 r1b {5,7};    // agregate initialization

struct Ratio2{
    int num, den;
    Ratio2(int c, int z): num(c), den(z) {}
};
Ratio2 r2a = {3,4}; //! Calls constructor Ratio2(int c, int z)
Ratio2 r2b {3,4};   //! Calls constructor Ratio2(int c, int z)

struct ArrWrapper
{
    const int arr [5]{3,4,4,5,6};
    const int arr2 [5]={3,4,4,5,6};
};
```

Види ініціалізації – шпаргалка

- Default initialization (немає ініціалізатора): **std::string str;**
- Value initialization (порожній ініціалізатор): **std::string str{};**
- Direct initialization (із вказаного списку аргументів): std::string str("text");
- Copy initialization (ініціалізуємо іншим об'єктом[2]): **std::string str = "text";**
- List initialization (за допомогою "списку у фігурних дужках"): std::string str{'t', 'e', 'x', 't'};
- Aggregate initialization (почленна ініціалізація агрегату): char cstr[5] = {'t', 'e', 'x', 't'};
- Reference initialization: char& ch = cstr[0];

При тому, list initialization має дві еквівалентні (до C++17 – див. далі) форми:

- Direct list initialization: **T object {arg1, arg2, ...};**
- Copy list initialization: T object = {arg1, arg2, ...};
- До C++17 в обох випадках вважалося, що викликається конструктор із аргументом initializer_list!

[2] Але пам'ятаємо про copy elision.

- Приклад винятку, коли узагальнена ініціалізація, {...}, не ідеальна: вона надає перевагу конструктору із списком ініціалізації над іншими придатними[3]. Тому, якщо потрібно скористатися іншим конструктором, слід використати круглі дужки – "функціональну ініціалізацію".

```
class VictimBase{
        .....
    explicit VictimBase(int v=0);
    VictimBase( initializer_list <int> l);
};

// VictimBase vx = {5}; //! Not Allowed because of explicit
//!Attention!
VictimBase vy{5}; //! initializer_list constructor called!
VictimBase vx(5); //! "Ordinary" constructor called!

VictimBase* dv1 = new VictimBase[5]{VictimBase{1},VictimBase{2},
                    VictimBase{3},VictimBase{2},
                    VictimBase{1}};
//new VictimBase[5] {1, 2, 3, 2, 1} does not work because
// of " explicit "
```

[3]Такими, які можна викликати із даними аргументами.

- При чому, іноді йому надається перевага, навіть якщо його використання породжує помилку компіляції і є інший конструктор, який точно співпадає по аргументах.

```
struct Cmb3{
    Cmb3(int, int, double) { cout << "1" << endl; };
    Cmb3( initializer_list <int>) { cout << "2" << endl; };
    Cmb3(string s) { cout << "3" << endl; };
    Cmb3(string s, string s2) { cout << "4" << endl; };
};
// Cmb3 cc5{ 2, 2, 3.5 }; // Error --- narrowing!
Cmb3 cc6{ "abc" };              // OK...
Cmb3 cc7{ "abc", "xxx" }; // OK...
struct Cmb4{
    Cmb4(double value, double uncertainty) {}
    Cmb4(std:: initializer_list <std:: string > values) {}
};
Cmb3 cc8{ 1, 2 };          // OK...
```

Покращення в C++17

- На загал, правило наступне: спочатку розглядаються конструктори із першим чи єдиними аргументом — списком ініціалізації. Якщо вони не підходять, тоді "примірятися" до списку в фігурних дужках починають всі конструктори.
- Така надмірна "важливість" конструктору із списку ініціалізації – проблемна, детальніше див. главу 7 книги "Efficient Modern C++" та `http://en.cppreference.com/w/cpp/language/list_initialization`.
- В новому стандарті, C++17 цю проблему спробували вирішити.

Покращення в C++17

- В C++17 внесли зміни щодо вибору конструктора.
- Copy list initialization все ще завжди використовує конструктор із initializer_list, а ось Direct list initialization змінило правила. Фігурні дужки із одним аргументом викличуть конструктор, що його приймає, а із більшою кількістю – заборонені, не скомпілюються:

```
VictimBase va = {42};   // Було б, якби не explicit :
                        // VictimBase( initializer_list <int>)
VictimBase vb{42};      // VictimBase(int)
VictimBase vc = {1, 2}; // VictimBase( initializer_list <int>)
VictimBase vd {1, 2};   // Error!
```

- Працює це завдяки зміненим правилам auto:

```
auto z5 = { 5, 6 };      // std :: initializer_list <int>
auto z6 = { 7, 8.0 };    // Помилка —— тип вивести не можна
auto z7{ 9, 10 };        // Помилка в C++17
auto z8 = { 11 };        // std :: initializer_list <int>
auto z9{ 12 };           // int
```

- Можна навіть так:

```
Ratio2a makeRatio2(int x, int y)
{
    return {x,y};
}

void printVector(const vector<int>& v)
{
    for(auto x : v )
        cout << x << ", ";
    cout << endl;
}
    .....
    Ratio2a r = makeRatio2(2,3);

    printVector ({5,6,4,3,2});
```

Узагальнюючи — 1

- Узагальнена ініціалізація може бути використана практично всюди.
- Для змінних фундаментальних типів повністю еквівалентна звичній "функціональній".
- Для агрегатів — масивів та структур **без**: користувацьких конструкторів, ініціалізації не-статичних членів, приватних чи захищених не-статичних членів, базових класів та віртуальних функцій, узагальнена ініціалізація працює звично — послідовно ініціалізує члени, зліва направо.
- Якщо ініціалізаторів для агрегату забагато, буде помилка.
- Якщо ж замало – решта ініціалізуються нулем або конструктором по замовчуванню, за відсутністю такого — почленно.

```
struct Ratio1{      int num, den;          };
struct DoubleRatio{
    Ratio1 a;
    Ratio1 b;
};
DoubleRatio dr1 {1};       // a.num=1, a.den=0, b.num=0, b.den=0
DoubleRatio dr1 {1, 2};    // a.num=1, a.den=2, b.num=0, b.den=0
DoubleRatio dr1 {1, 2, 3}; // a.num=1, a.den=2, b.num=3, b.den=0
```

Узагальнюючи — 2

- Для не-агрегатів — викликає конструктор. Тоді вимоги до кількості аргументів визначаються конструктором.
- Об'єднання так теж можна ініціалізувати, але лише перший член.
- Звуження заборонені[4]:

```
VictimBase vbn{3.14};//! Error!
int xz {1.23};          //! Error!
```

- Звуженням є перетворення у тип, не здатний представити всі можливі вихідного типу, за умови, що компілятор не може перевірити, чи конкретне значення вдасться зберегти без втрат.

```
struct Ci{
    short x;
    Ci(short x_): x{x_} {}
};

Ci a{5};              //! OK
// Ci b{5555555}; //! Error -- narrowing
// Ci c{x};          //! Error -- narrowing
```

[4]На жаль, GCC 4.9 компілює такий код із попередженням, GCC 5.2 — перший рядок вважає помилкою, але про другий лише попереджає. MSVC — дає помилку компіляції в обох випадках.

Узагальнюючи — 3

- Зазвичай узагальнену ініціалізацію можна використовувати і з "дорівнює" і без нього:

```
int x = {11};
int y {11};

Ratio2 r2a = {3,4}; //! Calls constructor Ratio2(int c, int z)
Ratio2 r2b {3,4};   //! Calls constructor Ratio2(int c, int z)
```

- Але є винятки. "=" **заборонено** використовувати:
- Якщо відповідний конструктор є explicit.
- Для ініціалізації членів класів їх конструктором:

```
struct Ipi{
    int x{3};                  // OK
    int y = {3};               // OK
    Ratio2 rx{1,2};            // OK
    // Ratio2 rx = {3,4};      // Error!
};
```

- **Рекомендація:** користуйтеся узагальненою ініціалізацією без символу "=".

initializer_list<T>

- Є спеціальний тип, initializer_list<T>, із `<initializer_list>`, який надає функціям (включаючи конструктори) доступ до ініціалізації не-агрегатів послідовністю елементів, так як ініціалізуються агрегати.
- Він не є повноцінним контейнером, абощо — лише інтерфейс до списків ініціалізації. Фізично може являти собою пару вказівників або вказівник і розмір. Наступний код:

```
initializer_list <double> il {1,2,3,4,4,56,6,7,78,8};
cout << sizeof(il) << endl;
```

  виведе 8 для x86-32 і 16 для x86-64 і для GCC і для MSVC.
- Містить: конструктор по замовчуванню (але сам компілятор створює не-порожні списки ініціалізації за потреби — див. приклади вище), методи size(), begin() та end(), які надають доступ до його елементів. Також, існують відповідні спеціалізації незалежних функцій begin() та end().
- Всі елементи — однотипні.
- Значення, передані `{...}` копіюються в певну (не визначену) ділянку пам'яті. Змінювати відповідний масив ініціалізаторів не можна! Подробиці див. `http://en.cppreference.com/w/cpp/utility/initializer_list`.

- Для зручності має також типовий набір typedef-ів: value_type, reference, const_reference, size_type, iterator, const_iterator.
- Методів rbegin() та rend() немає, але C++14 надає незалежні функції rbegin() та rend().
- Стандартна бібліотека завжди передає цей об'єкт по значенню.
- Приклад:

```
class dynamic_array{
        .....................
    dynamic_array( std:: initializer_list <T> l ): memory_( l.size() ) {
        for( auto &x : l )
            push_back(x);
    }
     iterator insert ( iterator pos, std :: initializer_list <T> l){
        return insert ( pos, l.begin (), l.end() );
    }
    template<class ForwardIter>
     iterator insert ( iterator pos, ForwardIter ibeg, ForwardIter iend)
     {...}
};
```

- Для підтримки конструкцій виду:

  int x = ...
  std :: vector<**int**> v = {x, x, x};

  стандарти, по C++23 включно, вважають, що об'єкт, на який посилається initializer_list знаходиться **в стеку**, що, для великих об'єктів може бути проблемою.
- Зокрема, таке можливо за використання #embed з C++26 – що дозволяє вставляти цілі двійкові файли в код.
- В статті "P1967 #embed and D2752 "Static storage for initializer_list" are now on Compiler Explorer" автор пропонує модифікацію стандарту, яка вирішує цю проблему: "Static storage for braced initializers".

- В списку ініціалізації може бути вкладений список:

```
struct Cmb1{
    Cmb1(int x, int y,   initializer_list <double> z){}
};

Cmb1 cc{ 1, 2, {5,5,5} };
```

- Можливим є "шаблонне" використання.

```
struct Cmb2{
    template <typename T>
    Cmb2( initializer_list <T> z){} // Cannot be local

    template <typename T>
    void fn(  initializer_list  <T> z){}
};

    Cmb2 cc2{1,3,4};
    Cmb2 cc3{'c','z','d'};
    Cmb2 cc4{Ratio2{1,2},Ratio2{3,4},Ratio2{5,6}};

    cc2.fn ({1,3,4});
    cc2.fn({'c','z','d'});
    cc2.fn({Ratio2{1,2},Ratio2{3,4},Ratio2{5,6}});
```

Structured Bindings – структуровані прив'язки, C++17+

- В C++17 було додано дуже зручну конструкцію:

```
std::tuple<int, double, std::string> fn1(){
    return std::make_tuple(5, 3.14, "abc"s);
}

auto [qu2, pi2, descr2] = fn1();
```

При цьому буде створено три змінні відповідних типів.

- Працює і для std::pair та масивів:

```
std::tuple<int, double, std::string> fn1(){
    return std::make_tuple(5, 3.14, "abc"s);
}

auto [qu2, pi2, descr2] = fn1();

std::pair<int, std::string> fn2(){ ... }
auto [qu2a, descr2a] = fn2();

int arr[2] = {1,2};
auto [x, y] = arr;
```

- Також працює для довільних структур чи класів, всі нестатичні члени яких є публічними:

```
struct Data
{
    int x;
    double y;
    std::string z;
};

Data fn3(){ ... }

auto [qu2b, pi2b, descr2b] = fn3();
```

- Раніше подібне можна було робити лише за допомогою std::tie() і для змінних, що вже існують:

```
int qu;
double pi;
std::string descr;

std::tie(qu, pi, descr) = fn1();
```

При тому tie() створює кортеж посилання (lvalue-посилань) на відповідні змінні.

- Як і для звичного auto, є можливість керувати "референсністю"отриманих змінних:

```
int  arr[2] = {1,2};
auto [x, y] = arr;        // Тип x та y -- int
auto& [xr, yr] = arr;     // Тип xr та yr -- int&,
                          // xr посилається на arr[0]
const auto& [xcr, ycr] = arr; // Тип xcr та ycr -- const int&
```

- Зокрема, одне із дуже зручних використань:

```
std::map<std::string, int> wc;
        .....................
for( const auto& [key,val]: wc){
     std::cout << key << " : " << val << std::endl;
}
```

- Іноді структурні прив'язки хотілося б використовувати і для класів, не всі члени яких є публічними. Це можна зробити:

```
class t_ratio_t{
    int num, den;
public:                 .....................................
    template <std::size_t N>
    decltype(auto) get() const {
        if       constexpr (N == 0) return num;
        else if  constexpr (N == 1) return den;
    }
};
namespace std {
    template<>
    struct tuple_size<t_ratio_t>
            : std::integral_constant<std::size_t, 2> {
    };
    template<std::size_t N>
    struct tuple_element<N, t_ratio_t> {
        using type = decltype(std::declval<t_ratio_t>().get<N>());
    };
}
```

- Ще приклад:
`https://devblogs.microsoft.com/oldnewthing/20201015-00/?p=104369` – "How to add C++ structured binding support to your own types".

- Також можна створювати якісь екзотичні класи, які при структурній прив'язці "віддають" щось нетривіальне. Але будьте обережні, заплутаний, неочевидний код – зло.
- Додавати щось до std, в цілому, заборонено, але ця ситуація – важливий виняток. Див. також: `https://en.cppreference.com/w/cpp/language/extending_std`.
- Ще фокус – створити кілька змінних різного типу за раз:

```
auto [v1, v2, v3] = std::make_tuple(std::vector<int>{5,7,13},
                                    std::string{"abc"}, 17);
```

Він зручний, коли виконується ініціалізація в for чи if:

```
for (auto [iss, name] = pair(istringstream(head), string{});
           getline(iss, name); ){
    ..........
}
```

Приклад взято тут: "3 Simple C++17 Features That Will Make Your Code Simpler" – там ще багато цікавого.

- Детальніше див. "Structured binding declaration" на cppreference. Текст там достатньо важко читати, але якщо розібратися і звикнути – прояснює деякі нюанси.

Трішки про механізми

- Коли виконується:

  auto [a, b, c] = ...;
  auto& [a, b, c] = ...;
  const auto& [a, b, c] = ...;

  і т.д., неявно створюється змінна-кортеж, яка потім "розпаковується" в a, b і c.

◇ Тобто:

  auto [a, b, c] = some_var;

  створює невидиму змінну, умовно, __abc, так що "a" буде транслюватися у "__abc.a", яка прив'язана до першого елементу some_var, "b" – у "__abc.b", для другого тощо.

◇ "Прив'язана" тут схоже на посилання, reference, але ним не є.

◇ Модифікатори &, const, volatile стосуються саме її, умовної "__abc.a"[5] – не елементів a, b, c, а цієї невидимої змінної.

- Отримані змінні, хоча і поводяться як звичайні, фактично є посиланнями на вихідні дані.

[5] Як і для лямбда-функцій, її ім'я чи тип недоступні, тому потрібен auto.

C++20

- В C++20 дозволили разом із structured binding використовувати static та thread_local.

```
static auto [a, b, c] = ....;
thread_local auto [a, b, c] = ....;
```

- Також, до C++20 такі змінні не можна було захоплювати у лямбда-функції.

C++26

- В C++26 запропоновано підтримку structured binding в умовних конструкціях та циклах

```
for (auto [index, value] : views::enumerate(vec)) {
    println("{}: {}", index, value);
    ...
}
```

```
if (auto [to, ec] = std::to_chars(p, last, 42)) {
    auto s = std::string(p, to);
    ...
}
```

◇ Спершу відбувається перевірка умови, потім – прив'язування. Тобто:

```
if (auto [to, ec] = .....)
```

перетворюється в:

```
if (auto __hiddenname = .....; auto [to, ec] = __hiddenname)
```

◇ "Structured binding declaration as a condition" – подробиці.

C++26

- Додано можливість окремо анотувати атрибутами окремі елементи прив'язки:

```
auto [a, b [[maybe_unused]], c] = ....;
```

- Додано constexpr structured binding – можливість обчислювати їх під час компіляції:

```
constexpr auto [a, b, c] = std::tuple(1, 2, 3);
```

- Додано можливість створювати пакети аргументів:

```
std :: tuple<X, Y, Z> f();
auto [... xs] = f();
auto [x, ... rest] = f();
auto [x,y,z, ... rest] = f(); // rest is an empty pack
auto [x, ... rest, z] = f(); // rest is a pack of length one
```

◇ Однак, поки (?), лише в шаблонах – щоб не ускладнювати реалізацію (приклад з відповідного документу):

```
template <class P, class Q>
auto dot_product(P p, Q q) {
    // no indirection !
    auto&& [...p_elems] = p;
    auto&& [...q_elems] = q;
    return (... + (p_elems * q_elems));
}
```

◇ Звичайно, це можна обійти (приклад звідси):

```
// auto [... args] = std::tuple(1, 2, 3); -- помилка поки
        auto val = std::tuple(1, 2, 3);

    [&](auto& arg) {
        auto [... args] = arg;
    }(val);
```

◇ Кажуть, поки немає повноцінної рефлексії в C++, це дуже спрощує реалізацію її ерзац-замінників.

Designated Initializers (DRAFT!) – C++20

- C++20 додав можливість ініціалізувати структури, вказуючи імена полів.
- При тому, порядок (на відміну від C99) має бути тим же, що й в оголошенні, але можна пропускати.
- Приклад:

```
struct ratio_t {
    int Num = 0;
    int Den = 1;
};

ratio_t r1 { .Num = 10, .Den = 20 };
ratio_t r2 { .Num = 11 };
ratio_t r3 { .Den = 22 };
```

- Гарний оглядовий есей на тему: "Designated Initializers, the best feature of C++20".

- В C++29 додали Designated Initializers для базових класів:

```
struct A     { int a; };
struct B : A { int b; };

B{{1}, 2};        // OK in C++17
B{1, 2};          // OK in C++17

B{.a=1, .b=2}     // currently OK in C++29
B{{.a=1}, .b=2}   // currently OK in C++29
B{.a{1}, .b{2}}   // currently OK in C++29
```

- Хороший огляд ініціалізації в C++: `https://habr.com/ru/company/jugru/blog/469465/`. Єдине, про неможливість використовувати списки ініціалізації із шаблонами – як мінімум, перебільшення. Ну і, в цілому, гуру C++ вважають, що створювати об'єкт слід фігурними дужками, а не круглими.
- Ілюстрація звідти: `https://hsto.org/webt/bp/rd/ow/bprdow1rk6jtw5fzpm3bffzirps.gif`
- Ще один, із тією ж ілюстрацією: `http://mikelui.io/2019/01/03/seriously-bonkers.html`.

Додаткові посилання

- Деякі граничні нюанси описано тут: `https://blog.feabhas.com/2019/04/brace-initialization-of-user-defined-types/`.
- Трохи про механізми – тут: `https://habr.com/ru/company/playrix/blog/465181/`.

Initialisation in C++17

Version 2.1 – Copyright (c) 2019 Timur Doumler

	Default init `;`	**Copy init** `= value;`	**Direct init** `(args);`	**Value init** `();`	**Empty braces** `{};  = {};`	**Direct list init** `{args};`	**Copy list init** `= {args};`
Built-in types	Uninitialised. Variables w/ static storage duration: Zero-initialised	Initialised with value (via conversion sequence)	1 arg: Init with arg >1 arg: Doesn't compile	Zero-initialised	Zero-initialised	1 arg: Init with arg >1 arg: Doesn't compile	1 arg: Init with arg >1 arg: Doesn't compile
auto	Doesn't compile	Initialised with value	Initialised with value	Doesn't compile	Doesn't compile	1 arg: Init with arg >1 arg: Doesn't compile	Object of type std::initializer_list
Aggregates	Uninitialised. Variables w/ static storage duration: Zero-initialised***	Doesn't compile	Doesn't compile (but will in C++20)	Zero-initialised***	Aggregate init**	1 arg: implicit copy/move ctor if possible. Otherwise aggregate init**	1 arg: implicit copy/move ctor if possible. Otherwise aggregate init**
Types with std::initializer_list ctor	Default ctor	Matching ctor (via conversion sequence), explicit ctors not considered	Matching ctor	Default ctor	Default ctor if there is one, otherwise std::initializer_list ctor	std::initializer_list ctor if possible, otherwise matching ctor	std::initializer_list ctor if possible, otherwise matching ctor****
Other types with no user-provided* default ctor	Members are default-initialised	Matching ctor (via conversion sequence), explicit ctors not considered	Matching ctor	Zero-initialised***	Zero-initialised***	Matching ctor	Matching ctor****
Other types	Default ctor	Matching ctor (via conversion sequence), explicit ctors not considered	Matching ctor	Default ctor	Default ctor	Matching ctor	Matching ctor****

*not user-provided = not user-declared, or user-declared as `=default` *inside* the class definition
**Aggregate init copy-inits all elements with given initialiser, or value-inits them if no initialiser given
***Zero initialisation zero-initialises all elements *and initialises all padding to zero bits*
****Copy-list-initialisation considers explicit ctors, too, but doesn't compile if such a ctor is selected

